K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

ADA LAB SHEET - 1

COURSE CODE - ENCS256

NAME : Tarun Yadav

COURSE : B.TECH CSE(FSD)

ROLL NO : 2401350015

GITHUB LINK :

TarunYadav121/2401350015_ADA

Title:

Sorting a Set of Numbers Using Various Algorithms and Analyzing Their Efficiency .

Introduction :

Sorting algorithms are fundamental in computer science, used to arrange data in a specific order to enhance efficiency and accessibility. This assignment focuses on implementing various sorting techniques to organize numbers into ascending or descending order. By utilizing the step-count method, we will calculate the time complexity of each algorithm to analyze their performance. Input data will be taken from a table and operations will be repeated with varying input sizes (10, 20, 30, 40 numbers). The study includes examining best-case, worst-case, and average-case scenarios, culminating in visualizing results through graphs to interpret computational trends effectively.

Objective:

The objective of this assignment is to implement multiple sorting algorithms, sort a given set of numbers in ascending and descending order, and analyze their performance using the step-count method. Additionally, students will examine the best-case, worst-case, and average-case scenarios and visualize the results through graphs.

CODE:

```
import java.util.*;

public class LabSheet1Sorting {

    // ---------- Bubble Sort ----------
    public static long bubbleSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;
        boolean swapped;

        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                comparisons++;  // comparison count
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break; // optimization
        }
        return comparisons;
    }

    // ---------- Selection Sort ----------
    public static long selectionSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                comparisons++;  // comparison count
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
```

```
                }
            }
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
        return comparisons;
    }

    // ---------- Insertion Sort ----------
    public static long insertionSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0) {
                comparisons++;  // comparison count
                if (arr[j] > key) {
                    arr[j + 1] = arr[j];
                    j--;
                } else {
                    break;
                }
            }
            arr[j + 1] = key;
        }
        return comparisons;
    }

    // ---------- Generate Random Array ----------
    public static int[] generateRandomArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(1000);
        }
        return arr;
    }

    // ---------- Generate Sorted Array (Best Case) ----------
```

```java
    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    // ---------- Generate Reverse Sorted Array (Worst Case) ----------
    public static int[] generateReverseArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = n - i;
        }
        return arr;
    }

    // ---------- Main ----------
    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int size : sizes) {

            System.out.println("================================");
            System.out.println("Input Size: " + size);
            System.out.println("================================");

            // ---------- Best Case ----------
            int[] best = generateSortedArray(size);

            System.out.println("BEST CASE (Sorted Input)");
            System.out.println("Bubble Sort Comparisons: " +
bubbleSort(best.clone()));
            System.out.println("Selection Sort Comparisons: " +
selectionSort(best.clone()));
            System.out.println("Insertion Sort Comparisons: " +
insertionSort(best.clone()));

            // ---------- Worst Case ----------
            int[] worst = generateReverseArray(size);

            System.out.println("\nWORST CASE (Reverse Sorted Input)");
```

```
            System.out.println("Bubble Sort Comparisons: " +
bubbleSort(worst.clone()));
            System.out.println("Selection Sort Comparisons: " +
selectionSort(worst.clone()));
            System.out.println("Insertion Sort Comparisons: " +
insertionSort(worst.clone()));

            // ---------- Average Case ----------
            int[] avg = generateRandomArray(size);

            System.out.println("\nAVERAGE CASE (Random Input)");
            System.out.println("Bubble Sort Comparisons: " +
bubbleSort(avg.clone()));
            System.out.println("Selection Sort Comparisons: " +
selectionSort(avg.clone()));
            System.out.println("Insertion Sort Comparisons: " +
insertionSort(avg.clone()));

            System.out.println("\n\n");
        }
    }
}
```

OUTPUT:

FOR INPUT SIZE 10:

```
PS C:\Users\Tarun\OneDrive\Desktop\ADA\LAB_SHEET_1>  & 'C:\Program Files\Ecli
ssages' '-cp' 'C:\Users\Tarun\AppData\Roaming\Code\User\workspaceStorage\1f6
Sorting'
================================
Input Size: 10
================================
BEST CASE (Sorted Input)
Bubble Sort Comparisons: 9
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 9

WORST CASE (Reverse Sorted Input)
Bubble Sort Comparisons: 45
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 45

AVERAGE CASE (Random Input)
Bubble Sort Comparisons: 39
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 23
```

FOR INPUT SIZE 20:

```
PS C:\Users\Tarun\OneDrive\Desktop\ADA\LAB_SHEET_1>  & 'C:\Program
ssages' '-cp' 'C:\Users\Tarun\AppData\Roaming\Code\User\workspaceS
Sorting'



==================================
Input Size: 20
==================================
BEST CASE (Sorted Input)
Bubble Sort Comparisons: 19
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 19

WORST CASE (Reverse Sorted Input)
Bubble Sort Comparisons: 190
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 190

AVERAGE CASE (Random Input)
Bubble Sort Comparisons: 184
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 127
```

FOR INPUT SIZE 30:

```
PS C:\Users\Tarun\OneDrive\Desktop\ADA\LAB_SHEET_1>  & 'C:\Program Files\Eclipse
ssages' '-cp' 'C:\Users\Tarun\AppData\Roaming\Code\User\workspaceStorage\1f6e0a1
Sorting'



==================================
Input Size: 30
==================================
BEST CASE (Sorted Input)
Bubble Sort Comparisons: 29
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 29

WORST CASE (Reverse Sorted Input)
Bubble Sort Comparisons: 435
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 435

AVERAGE CASE (Random Input)
Bubble Sort Comparisons: 399
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 214
```

FOR INPUT SIZE 40:

```
PS C:\Users\Tarun\OneDrive\Desktop\ADA\LAB_SHEET_1>  & 'C:\Program Files\Ecli
ssages' '-cp' 'C:\Users\Tarun\AppData\Roaming\Code\User\workspaceStorage\1f6e
Sorting'



==================================
Input Size: 40
==================================
BEST CASE (Sorted Input)
Bubble Sort Comparisons: 39
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 39

WORST CASE (Reverse Sorted Input)
Bubble Sort Comparisons: 780
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 780

AVERAGE CASE (Random Input)
Bubble Sort Comparisons: 714
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 400
```

FOR REVERSED SORTED AND RANDOM INPUT:

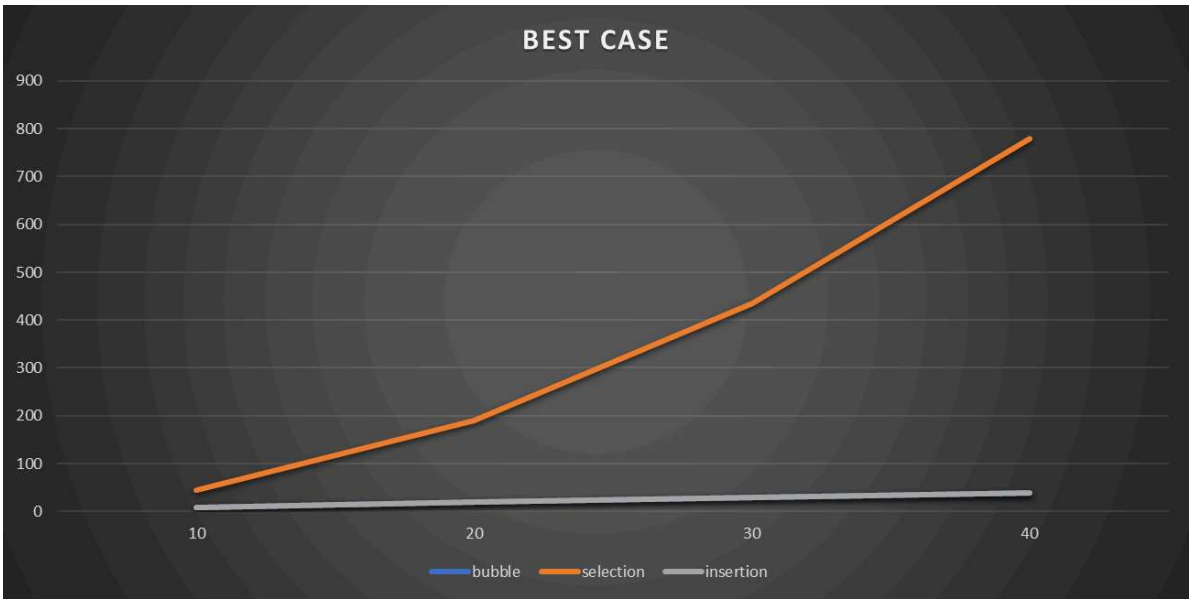
```
Insertion Sort Comparisons: 39

WORST CASE (Reverse Sorted Input)
Bubble Sort Comparisons: 780
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 780

AVERAGE CASE (Random Input)
Bubble Sort Comparisons: 774
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 421
```

TABLES:

BEST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	9	45	9
20	19	190	19
30	29	435	29
40	39	780	39

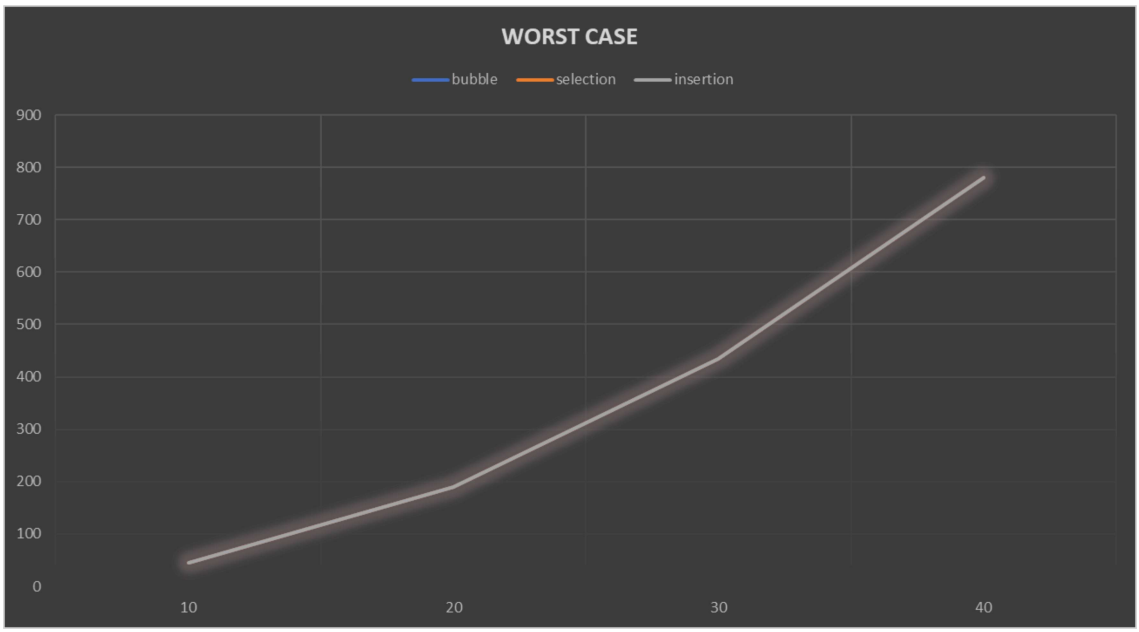
GRAPH



WORST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	45	45	45
20	190	190	190
30	435	435	435
40	780	780	780

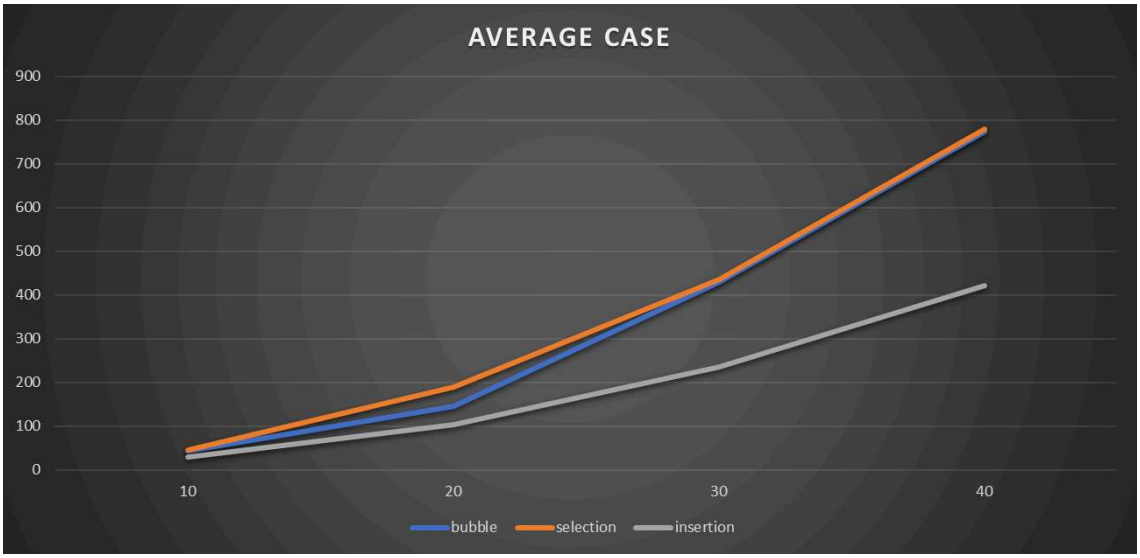
GRAPH



AVERAGE CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	44	45	30
20	145	190	104
30	429	435	236
40	774	780	421

GRAPH



Observation From Graphs:

From the graphs and comparison count tables, we observe that:

1. **Selection Sort** shows almost the same number of comparisons in best, average, and worst cases.
   This is because it always scans the entire unsorted portion of the array regardless of input order.
   Hence, its time complexity remains **O(n²)** in all cases.

2. **Bubble Sort** performs very efficiently in the best case due to the optimization (early stopping when no swaps occur).

Therefore, its best-case complexity becomes **O(n)**.
However, in average and worst cases, it performs nearly **$O(n^2)$** comparisons.

3. **Insertion Sort** performs efficiently for already sorted data, showing linear growth in the best case (**O(n)**).
   In the worst case, it performs **$O(n^2)$** comparisons, similar to Bubble and Selection Sort.

4. From the average-case graph, Insertion Sort performs better than Bubble Sort for larger inputs.